SAP ORDER-TO-CASH GRAPH QUERY SYSTEM

Functional Requirements · Architecture · Data Flow · Guardrails · Upgrade Paths

Version 2.0 (Final) | 2026

Attribute	Value
Document Title	SAP O2C Graph Query System — Complete Technical & Requirements Documentation
Version	2.0 Final — Functional Requirements + Architecture + Data Flow + Guardrails
Stack	Python 3.11 · FastAPI · SQLite · Groq LLaMA-3.3-70b · Vis.js · Vanilla HTML
Status	Production-ready with documented limitations
Audience	Engineering, Architecture Review, Product Stakeholders, Evaluators

1. Executive Summary

The SAP O2C Graph Query System transforms raw SAP CSV exports into an AI-powered, graph-backed analytical interface. Business users type plain-English questions and receive grounded, data-backed answers in seconds — every response is derived from live SQLite queries, not LLM memory.

1.1 What the system does

- Ingests 19 SAP CSV tables at startup into a local SQLite database with ISO date normalisation
- Constructs a knowledge graph with 8 entity node types and typed relationship edges
- Exposes a natural language chat interface backed by Groq LLaMA-3.3-70b
- The LLM generates SQL guided by an injected schema + join dictionary, executes it against real data, and returns a grounded plain-English answer
- Renders an interactive Vis.js graph — schema overview on load, per-entity subgraph after each query
- Applies 4-layer SQL security guardrails: domain classifier, system prompt, sqlparse AST validator, whitelisted tables

1.2 Business problems solved

Business question	How the system answers it
Which products generate the most billing documents?	LLM generates GROUP BY SQL across billing_document_items joined to products
Trace the full O2C flow for a sales order	7-table JOIN chain via injected join dictionary, result highlighted in graph
Find deliveries that were never billed	LEFT JOIN IS NULL between outbound_delivery_headers and billing_document_headers
Which invoices are unpaid past 30 days?	Date arithmetic on billing + payment tables with ISO-normalised date columns
Broken or incomplete flows	Multiple LEFT JOIN patterns detecting missing links at each O2C stage

2. Functional Requirements

Five functional requirements govern this system. Each is described below with its precise definition, how it is implemented, and what it means in practice.

FR1 Graph Construction

- Ingest the complete SAP dataset and construct a property graph representation in memory and on disk
- Define nodes for each business entity type: Customer, Sales Order, Sales Order Item, Delivery, Delivery Item, Billing Document, Billing Item, Journal Entry, Payment, Product, Plant
- Define typed edges representing real SAP relationships: Customer → places → Sales Order; Sales Order → contains → Sales Order Item; Sales Order Item → references → Delivery Item (composite key join); Delivery Item → part of → Delivery Header; Delivery Header → billed via → Billing Document; Billing Document → posted to → Journal Entry; Journal Entry → cleared by → Payment; Sales Order Item → is for material → Product; Delivery → ships from → Plant
- The graph model must reflect how SAP data actually connects — not just header-level IDs but item-level composite keys (SalesOrder + SalesOrderItem = ReferenceDocument + ReferenceDocumentItem)
- NODE_CONFIG dictionary maps 8 table names to id_col, label_col, color, and group
- EDGE_MAP list of tuples (src_table, src_col, tgt_table, tgt_col, relationship_label) drives both graph construction and subgraph fetching

FR2 Graph Visualization

- Build a browser-based interface that allows users to explore the graph interactively
- Support expanding nodes: clicking a node in the graph triggers a subgraph fetch (GET /graph/subgraph?entity_id=X) that loads connected entities up to 2 hops away
- Support inspecting node metadata: clicking any node shows a popup panel with all row fields from the underlying table (SalesOrder, NetValue, SoldToParty, BillingDate, etc.)
- Support viewing relationships: edges are labelled with the relationship type; color encodes entity group (customer=cyan, order=purple, delivery=amber, invoice=red, journal=green, payment=pink)
- Default view on page load is a Schema Overview — 8 group-level nodes and their typed edges — not a 9,500-node hairball
- After any NL query, the graph automatically switches to a Trace View showing only nodes connected to the queried entity
- A 'Schema View' toggle resets the graph to the overview
- Implementation uses Vis.js Network (CDN) with forceAtlas2Based physics layout

FR3 Conversational Query Interface

- Build a chat interface that accepts natural language questions about the O2C dataset
- Translate queries into SQL using Groq LLaMA-3.3-70b guided by an injected schema string and a Join Dictionary documenting all 10 O2C join paths
- All responses must be grounded in the dataset — the LLM receives actual SQLite query results (up to 20 rows as JSON) before generating its final plain-English answer

- The system uses a two-call pattern: Call 1 generates SQL + initial reasoning; Call 2 receives real result rows and produces the final grounded answer referencing specific document IDs and amounts
- Every response includes: the SQL generated, validation status, row count, execution time, tables used, and the first 10 result rows as an HTML table
- Conversation memory: last 6 messages (3 turns) are included in every request for follow-up context
- Highlighted entity IDs from answers are automatically traced in the graph panel

FR4 Example Queries — System Must Answer

- FR4a — Products with highest billing document count: SQL joins billing_document_items to products, groups by Material, orders by COUNT(BillingDocument) DESC, returns top 10 with product descriptions
- FR4b — Full O2C flow trace: Given a SalesOrder ID, SQL traverses the full 7-table chain (SO → SO Items → Delivery Items → Delivery Headers → Billing Headers → Journal Entries → Payments) and returns all linked document IDs with dates and statuses
- FR4c — Broken/incomplete flows: LEFT JOIN IS NULL patterns detect: (a) deliveries with no billing document, (b) billing documents with no delivery source, (c) journal entries with no clearing payment, (d) sales orders with no delivery at all
- Additional supported queries: unpaid invoices past N days; revenue by customer/plant/material; cancellation rate by billing type; average O2C cycle time (order date to payment date); orders with partial delivery only
- All queries must return results within 2 seconds for typical dataset sizes (up to 100k rows per table)

FR5 Guardrails — Domain Restriction

- Layer 1 — Backend domain classifier: before calling the LLM, a keyword-based check rejects questions clearly outside the O2C domain (general knowledge, creative writing, coding help unrelated to the system)
- Layer 2 — System prompt restriction: the LLM system prompt includes an explicit domain restriction: 'You ONLY answer questions about the provided dataset. If asked anything outside this domain, respond with exactly: This system is designed to answer questions related to the provided dataset only.'
- Layer 3 — SQL validator (sqlparse AST): validates that the generated SQL is a single SELECT statement, strips all comments before analysis (prevents comment-injection bypasses), rejects any DDL/DML tokens, and whitelists only the 19 known O2C tables
- Layer 4 — LIMIT enforcement: enforce_limit() strips any LLM-generated LIMIT and replaces it with a hard backend LIMIT 200 regardless of LLM output
- Layer 5 — Response validation: if SQL returns 0 rows, the system flags this and suggests rephrasing rather than letting the LLM hallucinate an answer
- Examples of correctly rejected prompts: 'Write me a poem', 'What is the capital of France?', 'Help me code a React app', 'SELECT * FROM users -- DROP TABLE payments'

3. End-to-End Data Flow

This section traces the complete journey of data through the system — from raw CSV files on disk to a grounded answer displayed in the browser. There are two distinct flows: the Startup Flow (runs once when the server starts) and the Query Flow (runs for every user question).

3.1 Startup flow — runs once at server launch

When python main.py is executed, the FastAPI lifespan() context manager fires before the server accepts any HTTP requests. The following 6 steps run in sequence:

1	**CSV Ingest** ingest_data()	glob.glob('./data/*.csv') discovers all 19 SAP export files Each file is read with pandas.read_csv(low_memory=False) Column names are stripped of whitespace and spaces replaced with underscores normalize_dates(df) detects date columns by name keyword ('Date','Time','Created','Posted','Cleared'), applies pd.to_datetime(infer_datetime_format=True), and reformats to YYYY-MM-DD string df.to_sql(table_name, conn, if_exists='replace', index=False) writes each table to SQLite (o2c.db) Output: 19 SQLite tables with ISO-normalised date columns, ready for SQL queries
2	**Schema Build** build_schema_string()	Queries sqlite_master to enumerate all table names Runs PRAGMA table_info(table) for each table to get column names and types Assembles a multi-line string: TABLE sales_order_headers:\n SalesOrder TEXT\n ... Output: DB_SCHEMA_STR — injected into every LLM system prompt
3	**Join Dictionary** JOIN_DICTIONARY constant	Hardcoded string constant documenting all 10 O2C table join paths with exact column names Covers: SO headers→items, SO items→Delivery items (composite key), Delivery items→Delivery headers, Delivery→Billing (via ReferenceDocument), Billing→Journal, Journal→Payment, SO→Customer, SO→Product Includes a full 7-table SQL chain template for end-to-end traces Output: JOIN_DICTIONARY string — injected alongside schema into every LLM prompt
4	**System Prompt** SYSTEM_PROMPT_TEMPLATE.format()	SYSTEM_PROMPT_TEMPLATE is formatted with {schema} = DB_SCHEMA_STR and {join_dict} = JOIN_DICTIONARY Final SYSTEM_PROMPT contains: domain restriction, full live schema, all join rules, response format rules, SQL rules (ISO dates, LIMIT 50, COALESCE for NULLs) Stored globally — prepended to every Groq API call

5	**Graph Build** build_schema_overview()	Creates 8 group-level nodes (one per entity type), not one per data row Creates edges between groups using EDGE_MAP relationship labels Stored in GRAPH_CACHE — served by GET /graph on every page load No data rows loaded into the graph at startup; subgraph data is fetched lazily per query Output: {nodes: [8 items], edges: [~7 items], mode: 'schema'}
6	**Server Ready** uvicorn running	FastAPI begins accepting HTTP requests on http://0.0.0.0:8000 Endpoints available: GET /graph, GET /graph/subgraph, POST /query, GET /health

3.2 Query flow — runs for every user question

When a user types a question and presses Send, the following 12 steps execute. Steps 1–3 happen in the browser; steps 4–12 happen in the FastAPI backend and Groq API:

1	**User Input** Browser → POST /query	User types a question in the chat panel (or clicks a quick-start pill) JavaScript builds payload: {question: 'Which products have most billing docs?', history: [last 6 messages]} fetch('http://localhost:8000/query', {method:'POST', body: JSON.stringify(payload)}) is called
2	**Domain Guard** Layer 1 guardrail	Backend checks question against O2C domain keywords before calling the LLM Off-topic queries (poetry, general knowledge, coding help) are rejected here without consuming an LLM token On-topic queries proceed to the LLM
3	**Groq Call 1** NL → SQL generation	Messages array assembled: [system_prompt] + last 6 history messages + user question groq_client.chat.completions.create(model='llama-3.3-70b-versatile', messages=messages) LLM receives: full live schema + all 10 join paths + domain restriction + format rules LLM output: plain text containing SQL wrapped in <sql>...</sql> tags plus initial reasoning
4	**SQL Extract** regex on <sql> tags	re.search(r'<sql>(.*?)</sql>', raw, re.DOTALL) extracts the SQL If no SQL tags found: raw response is returned as-is (guardrail response or domain rejection)
5	**SQL Validate**	sqlparse.parse(sql) — exactly 1 statement required

	is_safe_sql() — sqlparse AST	stmt.get_type() must be SELECT (rejects INSERT, UPDATE, DELETE, DROP, CREATE) sqlparse.format(sql, strip_comments=True) strips all SQL comments before token scan Token scan: any DDL/DML token not in {SELECT, FROM, WHERE, JOIN} blocks the query Table whitelist: all table names after FROM/JOIN must be in ALLOWED_TABLES (the 19 O2C tables) Returns (True, 'ok') or (False, 'reason') — blocked queries return error message to user
6	**LIMIT Enforce** enforce_limit()	Strips any LIMIT clause the LLM generated using regex Appends LIMIT 200 to the end of the SQL Backend always controls maximum row count — never trusted from LLM output
7	**SQLite Execute** conn.execute(safe_sql)	sqlite3.connect(DB_PATH) — connects to o2c.db Execution time measured: t0 = time.perf_counter() cur.execute(safe_sql) — runs the validated, limited SQL against the 19 ingested tables rows = [dict(r) for r in cur.fetchall()] — results as list of dicts exec_ms = round((time.perf_counter() - t0) * 1000, 2)
8	**Groq Call 2** Results → grounded answer	Second Groq call with extended messages: original context + Call 1 output + result rows User message: 'SQL returned N rows: [{...}]. Give final answer.' LLM produces plain-English answer referencing real document IDs, amounts, and counts from the data Entity IDs extracted from answer via regex: re.findall(r'\b[A-Z0-9]{6,20}\b', answer)
9	**API Response** JSON payload assembled	Return payload: {answer, sql, sql_valid, sql_blocked_reason, rows, row_count, exec_ms, tables_used, highlighted_ids} tables_used extracted from SQL via regex on FROM/JOIN keywords FastAPI returns JSONResponse to browser
10	**Chat Render** Browser renders answer	Answer text displayed in chat panel Collapsible SQL panel rendered: generated SQL, validation badge, row count, exec time, tables used as pills, Copy SQL button First 10 result rows rendered as an HTML table below the answer
11	**Subgraph Fetch** GET /graph/subgraph	If highlighted_ids.length > 0, browser calls GET /graph/subgraph?entity_id={highlighted_ids[0]}&depth=2 fetch_subgraph() walks EDGE_MAP up to 2 hops from the entity ID using parameterised SQLite queries (LIMIT 50 per hop)

	Returns {nodes, edges, mode: 'subgraph'} containing only entities connected to the queried document

12	**Graph Update** Vis.js dataset replace	Vis.js nodes and edges DataSets are replaced with the subgraph data Physics layout re-runs — graph now shows only the relevant O2C chain 'Schema View' button resets graph to the original 8-node overview (re-fetches GET /graph) User can click any node in the subgraph to inspect its full row metadata in a popup panel

3.3 Data flow summary

Step	Who acts	Input	Output	Where
CSV → SQLite	ingest_data()	19 .csv files	19 SQLite tables, ISO dates	Backend startup
Schema string	build_schema_string()	SQLite sqlite_master	Multi-line schema text	Backend startup
System prompt	SYSTEM_PROMPT_TEMPLATE	Schema + join dict	Formatted system prompt	Backend startup
Graph overview	build_schema_overview()	NODE_CONFIG + EDGE_MAP	8-node schema graph JSON	Backend startup
NL → SQL	Groq Call 1	System prompt + question	Raw text with <sql> tags	Per query
SQL extraction	regex	Raw LLM output	SQL string	Per query
SQL validation	is_safe_sql()	SQL string	(bool, reason)	Per query
LIMIT enforce	enforce_limit()	SQL string	SQL + LIMIT 200	Per query
SQL execute	SQLite	Validated SQL	List of result dicts + timing	Per query
Grounded answer	Groq Call 2	Context + result rows	Plain-English answer with IDs	Per query
API response	FastAPI	Answer + metadata	JSON to browser	Per query
Chat render	Browser JS	JSON payload	Answer text + SQL panel + table	Browser
Subgraph fetch	fetch_subgraph()	Entity ID + depth	Focused node/edge subgraph	Per query
Graph update	Vis.js	Subgraph JSON	Interactive trace view in browser	Browser

4. High-Level Architecture

The system follows a 5-layer architecture: Data → Ingestion → Intelligence → API → Presentation. Each layer has a single responsibility and communicates only with adjacent layers.

4.1 Architecture layers

Layer	Components	Technology
L1 — Data	19 SAP CSV files	*CSV flat files, ./data/ folder*
L2 — Ingestion	CSV reader, date normaliser, SQLite writer, schema extractor	*pandas 2.x, sqlite3 stdlib*
L3 — Intelligence	Join dictionary, LLM calls, SQL validator, LIMIT enforcer	*Groq API, LLaMA-3.3-70b, sqlparse 0.5*
L4 — API	REST endpoints: /graph, /graph/subgraph, /query, /health	*FastAPI 0.115, uvicorn, CORS*
L5 — UI	Vis.js graph panel, chat panel, SQL disclosure, subgraph switch	*Vanilla HTML/JS, Vis.js 9.x CDN*

4.2 Technical choices — why each decision was made

Decision	Choice	Why	Trade-off
LLM Provider	Groq (LLaMA-3.3-70b)	200+ tokens/sec, free tier, strong SQL generation, 128k context	Hosted; no offline mode
Database	SQLite (file-based)	Zero setup, stdlib, standard SQL, file survives restarts	Single-writer; not for concurrent prod use
Web Framework	FastAPI	Async, auto OpenAPI docs, Pydantic validation, minimal boilerplate	Overkill for a demo; Flask also works
Frontend	Vanilla HTML + JS	No build step, opens in browser directly, zero npm/webpack	No component reuse; framework needed to scale UI
Graph Library	Vis.js Network	Built-in physics, clustering, click events, CDN delivery	Physics heavy above 2k nodes
SQL Validation	sqlparse (AST-based)	Comment-injection immune, proper statement parsing	Doesn't catch all semantic SQL errors
Date Storage	ISO-8601 text in SQLite	String comparisons work correctly on ISO dates; consistent with LLM	No timezone support; loses timestamp precision
Graph Mode	Schema overview + lazy subgraph	Startup instant; traces are readable; no hairball	Subgraph fetch adds latency after each query
Join Approach	Hardcoded join dictionary	Eliminates join hallucination; paths are deterministic not learned	Manual update needed when schema changes

5. Graph Model — Nodes and Edges

5.1 Node types

Group	Source table	ID column	Color	Represents
customer	business_partners	BusinessPartner	#3DE8FF	B2B customer accounts
order	sales_order_headers	SalesOrder	#A78BFA	Sales order headers
delivery	outbound_delivery_headers	DeliveryDocument	#FFB347	Outbound shipment headers
invoice	billing_document_headers	BillingDocument	#FF6B6B	Invoice / billing documents
journal	journal_entry_items_accounts_receivable	AccountingDocument	#4FFFB0	AR accounting journal postings
payment	payments_accounts_receivable	ClearingDocument	#F472B6	Customer payment clearing docs
product	products	Product	#E8B06A	Material / product master records
plant	plants	Plant	#7A8094	Warehouse / plant locations

5.2 Edge types — key join paths

From	To	Relationship	Join key
sales_order_headers	sales_order_items	contains	SalesOrder = SalesOrder
sales_order_items	outbound_delivery_items	referenced by	SalesOrder+SalesOrderItem = ReferenceDocument+ReferenceDocumentItem
outbound_delivery_items	outbound_delivery_headers	part of	DeliveryDocument = DeliveryDocument
outbound_delivery_headers	billing_document_headers	billed via	DeliveryDocument = ReferenceDocument
billing_document_headers	journal_entry_items_ar	posted to	BillingDocument = BillingDocument

From	To	Relationship	Join key
journal_entry_items_ar	payments_accounts_receivable	cleared by	AccountingDocument = ClearingDocument
sales_order_headers	business_partners	placed by	SoldToParty = BusinessPartner
sales_order_items	products	is for material	Material = Product
outbound_delivery_items	product_plants	sourced from	Material+Plant = Material+Plant

6. Guardrails — Domain Restriction in Detail

The system applies five layers of guardrails to ensure responses are always grounded in the O2C dataset and never leak unsafe SQL or off-topic answers.

Layer	Name	Mechanism	What it blocks
1	Backend domain classifier	Keyword check on question before LLM call	General knowledge, creative writing, coding help, clearly off-topic requests
2	System prompt restriction	Explicit instruction in every LLM call: respond with canned message for off-topic	LLM answering from its own knowledge without querying the dataset
3	sqlparse AST validator	Strips comments, parses statement tree, checks type = SELECT, scans tokens	Comment injection, multi-statement attacks, DDL/DML, unknown tables
4	Backend LIMIT enforcement	enforce_limit() strips LLM LIMIT and appends LIMIT 200	LLM returning unbounded result sets; resource exhaustion
5	Empty result validation	If len(rows) == 0, system surfaces the empty result rather than allowing hallucinated answer	LLM fabricating data when query returns nothing

Example — rejected prompts

The following queries are correctly handled: 'Write me a haiku about SAP' → blocked by Layer 1 domain check. 'What is the GDP of Germany?' → blocked by Layer 2 system prompt. 'SELECT * FROM sales_order_headers -- DROP TABLE payments_accounts_receivable' → blocked by Layer 3 (comment stripped, only one SELECT detected but the comment contains DDL). 'SELECT SalesOrder FROM nonexistent_table' → blocked by Layer 3 (unknown_table). All blocked queries return a clear user-facing message explaining why the request was rejected.

7. Critical Fixes Applied — v2 Changes

Four critical fixes were identified during architecture review and applied in v2. Each addresses a specific failure mode in the original implementation.

Fix	Layer	Problem	Solution	Status
A — Join Dictionary	L3 Intelligence	LLM hallucinated join keys. Only table/column names were in the prompt; SAP composite key joins are not guessable from column names alone.	Added JOIN_DICTIONARY covering all 10 O2C paths + full chain template. Injected into every system prompt.	APPLIED
B — Date Normalisation	L2 Ingestion	SAP CSV exports dates in multiple formats (YYYYMMDD, DD.MM.YYYY, ISO). SQLite string comparisons on non-ISO dates silently return wrong or zero results.	Added normalize_dates() helper. Detects date columns by name, applies pd.to_datetime(), reformats to YYYY-MM-DD before to_sql().	APPLIED
C — Subgraph Fetching	L3 Graph	build_graph(limit=500) × 19 tables = 9,500 potential nodes. Vis.js physics collapses; graph becomes unreadable.	Replaced with build_schema_overview() (8 nodes) for startup. Added fetch_subgraph() + GET /graph/subgraph for per-query traces.	APPLIED
D — sqlparse Validator	L3 Security	String-matching validator ('drop' not in sql.lower()) bypassed by comment injection: SELECT * FROM t -- DROP TABLE users.	Replaced with sqlparse AST validator. Strips comments, checks type, scans tokens, whitelists tables. Added enforce_limit().	APPLIED

8. Known Limitations

8.1 Data limitations

L1 — Static snapshot	The system ingests CSV files once at startup. No CDC, no incremental load, no live SAP connection. Data freshness depends entirely on when the CSV was exported.
L2 — No schema drift detection	Column name mismatches between CSVs and NODE_CONFIG/EDGE_MAP cause silent query failures. No automatic detection or warning at startup.
L3 — Single-user SQLite	SQLite has a single write lock. Concurrent requests from multiple users produce 'database is locked' errors.

8.2 AI / LLM limitations

L4 — Residual hallucination risk	The join dictionary covers main O2C paths but cannot cover all possible multi-table queries. Unusual aggregations across 5+ tables may still produce incorrect SQL.
L5 — No retry on empty results	A valid but semantically wrong query returning 0 rows shows an empty result. The user must rephrase; no automatic retry is implemented.
L6 — Groq rate limits	The free Groq tier has RPM and TPM limits. Under heavy use, queries fail with a 429 error. No retry logic is currently implemented.

8.3 Security limitations

L7 — No authentication	The FastAPI backend has no auth layer. CORS policy allows all origins (*). Must be replaced with OAuth2/API keys for any production deployment.
L8 — sqlparse does not block all SQL attacks	The validator blocks DDL, multi-statement attacks, and unknown tables. It does not prevent time-based blind injection via CASE/WHEN or deeply nested subquery exhaustion.

9. Upgrade Paths

9.1 Database upgrade

Current	Upgrade	Why	Effort
SQLite	PostgreSQL	Multi-user concurrency, connection pooling, proper indexes, EXPLAIN ANALYZE for LLM query optimisation	Medium — change connection string + SQLAlchemy
SQLite	DuckDB	In-process like SQLite but columnar — analytical GROUP BY / SUM queries run 10–100x faster. Best for read-heavy OLAP.	Low — single pip install, same pandas API
SQLite	Snowflake / BigQuery	Petabyte scale, native SAP connector, auto-scaling, built-in BI integrations	High — requires cloud infra + cost management

9.2 LLM upgrade

Current	Upgrade	Why	Effort
Groq LLaMA-3.3-70b	Anthropic Claude Sonnet	Stronger reasoning for complex multi-table SQL; lower hallucination rate	Low — swap API call
Groq hosted	Local Ollama (llama3.2, qwen2.5-coder)	No API cost; no rate limits; data never leaves the machine — critical for sensitive SAP data	Medium — requires GPU or accept latency
Two-call pattern	Single-call structured output	Reduce latency 30–50% using JSON mode / function calling to get SQL + answer in one call	Medium — prompt + parser redesign

9.3 Full production architecture upgrade

Component	Demo (current)	Production upgrade	Key benefit
Auth	None	OAuth2 / JWT / Azure AD	Multi-user, role-based access control
Data refresh	Manual CSV restart	SAP OData API + change CDC	Real-time or near-real-time data
Backend	Single FastAPI process	Kubernetes + horizontal pod autoscale	Handle concurrent users
Queue	Synchronous Groq call	Celery / Redis queue	Non-blocking queries, retry logic
Monitoring	Console print()	Prometheus + Grafana / Datadog	Latency, error rate, LLM cost tracking
Frontend	Single HTML file	React + shadcn/ui	Component reuse, accessibility, testing

Component	Demo (current)	Production upgrade	Key benefit
Database	SQLite	PostgreSQL + read replica	Concurrent access, ACID guarantees

10. Validation Test Queries

Use these 8 queries after deployment to verify each functional requirement and applied fix:

#	Query	Tests	Expected result
1	How many sales orders are in the system and what is the total net value?	FR1 ingestion + basic SELECT + SUM	Returns row count and aggregated net value; fails if CSV ingestion broke
2	List the top 5 customers by number of sales orders placed.	FR3 NL→SQL + JOIN: sales_order_headers → business_partners on SoldToParty	5 customer names with order counts
3	Which products have the highest number of billing documents? Show top 10.	FR4a + Fix A: composite key join through items to billing	10 product IDs with billing counts; 0 rows = join dict not working
4	Trace the complete O2C flow for the first sales order in the system.	FR4b + Fix A: full 7-table chain SO → Payment	Single row with all document IDs; 0 rows = chain broken
5	Show all deliveries that were shipped but never converted to a billing document.	FR4c: LEFT JOIN IS NULL pattern	List of DeliveryDocument IDs with no matching billing record
6	Which billing documents were created after 2025-01-01 and have not been cleared?	Fix B: ISO date comparison + LEFT JOIN payments	Correct results; 0 rows with valid data = date normalisation failed
7	Write me a poem about supply chains.	FR5: domain guardrail	Canned rejection message; no LLM token consumed
8 (security)	SELECT * FROM sales_order_headers -- DROP TABLE payments_accounts_receivable	Fix D: sqlparse validator	Blocked; user sees 'forbidden' message; SQLite never executed

11. Deployment & Setup

11.1 Prerequisites

- Python 3.11+
- pip available
- Groq API key from console.groq.com (free tier sufficient for development)
- 19 SAP CSV files exported with exact filenames matching table names

11.2 Quick start — 6 commands

1. mkdir sap-o2c-graph && cd sap-o2c-graph
2. python -m venv venv && source venv/bin/activate (Windows: venv\Scripts\activate)
3. pip install fastapi uvicorn groq pandas openpyxl sqlparse
4. Copy all 19 CSV files into ./data/
5. echo 'GROQ_API_KEY=gsk_your_key_here' > .env
6. python main.py — server starts on http://0.0.0.0:8000, open index.html in browser

11.3 File inventory

File	Lines (approx)	Purpose	Critical functions
main.py	~350	FastAPI app, ingestor, schema builder, graph builder, SQL validator, Groq engine, all API endpoints	ingest_data, normalize_dates, build_schema_string, build_schema_overview, fetch_subgraph, is_safe_sql, enforce_limit, query_groq
index.html	~400	Complete single-file frontend: Vis.js graph, chat UI, SQL panel, subgraph switching	renderMessage, renderSqlPanel, toggleSql, loadGraph, loadSubgraph, sendQuery
requirements.txt	6	Python dependencies	fastapi, uvicorn, groq, pandas, openpyxl, sqlparse
.env	1	Groq API key	GROQ_API_KEY
o2c.db	Gen.	SQLite DB created at first startup from CSV files	All 19 tables with ISO-normalised dates
data/*.csv ×19	Varies	SAP export files — must be named exactly as table names	Source of all business data

Most common failure mode

Column name mismatch — if your CSV has 'Sales_Order' but NODE_CONFIG expects 'SalesOrder', queries silently fail. Diagnose with: pandas.read_csv('data/sales_order_headers.csv', nrows=1).columns.tolist(). Compare each column against the expected names in main.py and update NODE_CONFIG and EDGE_MAP accordingly.

12. Glossary

Term	Definition
O2C	Order-to-Cash — the end-to-end SAP business process from customer order placement through final payment receipt
LLM	Large Language Model — the AI model (LLaMA-3.3-70b) that translates natural language to SQL and results back to English
Groq	Cloud inference provider hosting the LLaMA model with very high throughput (~200 tokens/sec)
SQLite	Serverless, file-based SQL database built into Python's standard library; used as the query engine for all 19 ingested tables
FastAPI	Modern Python async web framework used for the REST API backend
Vis.js	JavaScript network graph visualisation library loaded from CDN; renders the O2C entity graph
sqlparse	Python library for SQL AST parsing — used in the security validator to prevent injection attacks
Join Dictionary	Hardcoded string in the system prompt documenting all SAP table join keys — prevents LLM from hallucinating join conditions
Schema Overview	Default graph view: 8 entity-type nodes (not individual data rows); loads at startup
Subgraph	Focused graph view showing only nodes connected to a queried entity — fetched per-query via /graph/subgraph
ISO-8601	Date format YYYY-MM-DD — used after normalisation so SQLite date comparisons work correctly
HARD_LIMIT	Backend-enforced LIMIT 200 appended to every SQL query regardless of LLM output
NODE_CONFIG	Python dict mapping table names to graph config: id column, label column, color, group
EDGE_MAP	Python list of (src_table, src_col, tgt_table, tgt_col, relationship_label) tuples defining graph edges
Two-call pattern	Architecture where Call 1 generates SQL and Call 2 receives result rows to produce a grounded answer
lifespan()	FastAPI startup/shutdown context manager that runs ingest, schema build, and graph build before the server accepts requests
CDC	Change Data Capture — a method for detecting and propagating incremental data changes from a source system (not currently implemented)
AST	Abstract Syntax Tree — the parsed tree representation of a SQL statement used by sqlparse for safe validation

SAP O2C Graph Query System — Final Technical Documentation v2.0

5 Functional Requirements · 14-step end-to-end data flow · 4 critical fixes · Architecture · Guardrails · Upgrade paths